

Support HTML modulaire

Master 2

6 jours

Java / Spring Boot

Jour 6 — Déploiement, cloud, gouvernance & architecture d'entreprise

Objectifs pédagogiques

Docker multi-stage, Kubernetes avancé, CI/CD GitHub Actions, IaC Terraform, gouvernance ADR, choix contextuels d'architecture.

Temps estimé

≈ 6h incluant démos, live coding et quiz

Prérequis

Jours 1 à 5 acquis

Table des matières

- 1. Conteneurs & Docker
- 2. Kubernetes avancé
- 3. CI/CD & IaC
- 4. Gouvernance & ADR
- 5. Urbanisation & choix
- 6. Clôture & évaluation
- Quiz de fin

1. Conteneurs et Docker

🧒 Mode enfant

Un conteneur c'est comme une boîte de transport standardisée : peu importe ce qu'il y a dedans, elle s'emboîte partout de la meme facon — su laptop, le serveur du client, le cloud. Plus de "ca marche chez moi mais pas en prod".

	VM	Conteneur
Isolation	Noyau + OS complet	Processus isolés (namespaces Linux)
Taille	Plusieurs Go	Quelques Mo à quelques centaines de Mo
Démarrage	Plusieurs minutes	Secondes
Portabilité	Limitée	Totale (Docker Engine)

1.1 Dockerfile basique

Junior

❌ DOCKERFILE NAIF — IMAGE INUTILEMENT LOURDE

• • • DOCKERFILE

```
FROM eclipse-temurin:21
COPY . /app
WORKDIR /app
RUN ./mvnw package
CMD ["java", "-jar", "target/catalog-service.jar"]
```

✓ DOCKERFILE OPTIMISÉ — MULTI-STAGE BUILD

```
• • • DOCKERFILE

# Stage 1 : Build (JDK + Maven)
FROM eclipse-temurin:21-jdk AS build
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn package -DskipTests -q

# Stage 2 : Runtime (JRE seul = image 3x plus petite)
FROM eclipse-temurin:21-jre AS runtime
RUN addgroup --system appgroup \
  && adduser --system --ingroup appgroup appuser
WORKDIR /app
COPY --from=build /build/target/catalog-service.jar app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", \
  "-XX:MaxRAMPercentage=75.0", \
  "-Djava.security.egd=file:/dev/./urandom", \
  "-jar", "app.jar"]
```

✓ Checklist Dockerfile production

- `.dockerignore` : exclure `target/`, `.git/`, `*.md`
- Utilisateur non-root (`USER appuser`)
- Image de base minimale (JRE != JDK)
- Jamais de secrets dans le Dockerfile
- Scan CVE : `trivy image catalog-service:latest`

• • • .DOCKERIGNORE

```
.git
target/
*.md
*.log
.env
.idea
**/*.class
```

2. Kubernetes — Orchestration avancée

👶 Mode enfant

Kubernetes c'est le chef d'orchestre de tes conteneurs. Tu lui dis "je veux 3 copies de ce service" et il s'en assure en permanence, même si l'un p — il le remplace automatiquement.

2.1 Deployment complet avec sondes et ressources

● Intermédiaire

```
• • • CATALOG-DEPLOYMENT.YAML

apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-service
  namespace: library
spec:
  replicas: 3
  selector:
    matchLabels:
      app: catalog-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1 # au max 1 pod indispo pendant le déploiement
      maxSurge: 1 # au max 1 pod supplémentaire pendant la transition
  template:
    metadata:
      labels:
        app: catalog-service
    spec:
      containers:
        - name: catalog-service
          image: registry.example.com/catalog-service:1.2.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "1000m"
              memory: "512Mi"
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
            - name: DATABASE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: catalog-db-secret
                  key: password
          startupProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            failureThreshold: 30
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            periodSeconds: 20
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
            periodSeconds: 10
            failureThreshold: 3
```

2.2 HPA — Horizontal Pod Autoscaler

● Expert

```
• • • CATALOG-HPA.YAML

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: catalog-hpa
  namespace: library
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: catalog-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # Scale si CPU moyen > 70%
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "100"
```

⚠️ Kubernetes Secrets != sécurisés par défaut

Les Secrets sont encodés en base64 dans etcd — pas chiffrés. En production : activer le chiffrement etcd ou utiliser HashiCorp Vault Injector / Se...

📖 Live Coding — Rolling update sans downtime

- Déployer `catalog-service:1.0.0` avec 3 réplicas
- Dans un terminal : `watch -n1 curl http://catalog/actuator/health`
- Mettre à jour l'image : `kubect1 set image deployment/catalog-service catalog-service=registry/catalog-service:1.1.0`
- Observer : `kubect1 rollout status deployment/catalog-service`
- Rollback : `kubect1 rollout undo deployment/catalog-service`

3. Stratégies de déploiement, CI/CD et IaC

Pratique	But	Avantage	Coût / Risque
Rolling	Remplacement progressif des pods	Pas de downtime, simple	2 versions en parallèle brièvement
Blue-Green	Bascule nette entre 2 environnements	Rollback instantané	Double coût infra
Canary	Exposition progressive (5% → 100%)	Limite le blast radius	Complexité routing + observabilité
Feature Flags	Découpler déploiement et activation	Rollback sans re-déploiement	Complexité du code

Blue-Green :

```

Load Balancer
|
+---> [Blue v1.0 - 3 pods] (production actuelle)
|
+---> [Green v1.1 - 3 pods] (nouvelle version testée)
-> Basculer 100% du trafic d'un coup -> Blue reste pour rollback instantané

```

Canary :

```

Load Balancer
|
+---> [Stable v1.0 - 90%]
|
+---> [Canary v1.1 - 10%] -> surveiller métriques -> augmenter progressivement

```

3.1 Pipeline CI/CD avec GitHub Actions

● Intermédiaire

```

• • • .GITHUB/WORKFLOWS/CATALOG-SERVICE.YML

name: CI/CD - catalog-service
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  IMAGE: registry.example.com/catalog-service
  JAVA_VERSION: '21'

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          java-version: ${ env.JAVA_VERSION }
          distribution: temurin
          cache: maven
      - name: Build and tests
        run: mvn verify -q
      - name: SonarQube analysis
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        run: mvn sonar:sonar

  docker-build:
    needs: build-and-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t $IMAGE:${ github.sha } .
      - name: Scan CVE (Trivy)
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${ env.IMAGE }:${ github.sha }
          exit-code: '1'
          severity: 'CRITICAL,HIGH'

```

```

- name: Push
  run: |
    echo "${{ secrets.REGISTRY_TOKEN }}" | docker login -u "${{ secrets.REGISTRY_USER }}" --password-stdin registry.example.com
    docker push $IMAGE:${{ github.sha }}

deploy-staging:
  needs: docker-build
  environment: staging
  runs-on: ubuntu-latest
  steps:
    - name: Rolling update staging
      run: kubectl set image deployment/catalog-service catalog-service=$IMAGE:${{ github.sha }} --namespace=staging
    - name: Smoke test
      run: |
        sleep 30
        curl -f https://catalog.staging.example.com/actuator/health

deploy-production:
  needs: deploy-staging
  environment: production # Approbation manuelle requise dans GitHub
  runs-on: ubuntu-latest
  steps:
    - name: Deploy canary 10%
      run: kubectl set image deployment/catalog-service-canary catalog-service=$IMAGE:${{ github.sha }} --namespace=production

```

3.2 Infrastructure as Code avec Terraform

● Expert

```

• • • MAIN.TF - CLUSTER GKE

terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
  backend "gcs" {
    bucket = "tfstate-library-prod"
    prefix = "catalog-service"
  }
}

resource "google_container_cluster" "library_cluster" {
  name                  = "library-cluster"
  location              = "europe-west1"
  remove_default_node_pool = true
  initial_node_count    = 1
}

resource "google_container_node_pool" "primary_nodes" {
  name        = "primary-pool"
  cluster     = google_container_cluster.library_cluster.name
  location    = google_container_cluster.library_cluster.location
  node_count  = 3
  node_config {
    machine_type = "e2-standard-2"
    disk_size_gb = 50
    labels      = { env = "prod" }
  }
  management {
    auto_repair = true
    auto_upgrade = true
  }
}

```

```
}  
}
```

✓ Bonnes pratiques IaC

- État Terraform dans un backend remote (GCS, S3) — jamais localement
- `terraform plan` dans la PR, `terraform apply` sur merge main uniquement
- Modules réutilisables par équipe (principe DRY)
- Secrets sensibles dans les variables CI, jamais dans `terraform.tfvars` versionné
- Tagger toutes les ressources (coût, équipe, environnement)

4. Gouvernance d'API, versioning et ADR

✓ Checklist gouvernance API

- Nommage cohérent des endpoints (`/api/v1/loans`) noms au pluriel
- Versioning explicite dans l'URL ou le header `Accept`
- Contrats OpenAPI publiés dans un portail développeur
- Dépréciation annoncée avec délai (ex. 6 mois) via header `Deprecation`
- Contract testing (Spring Cloud Contract, Pact)
- ADR pour chaque décision architecturale importante

4.1 Architecture Decision Record (ADR)

● Junior

Qu'est-ce qu'un ADR ?

Un document court qui capture une décision architecturale importante : le contexte, la décision, les alternatives considérées et les conséquences. C'est la mémoire vivante de l'architecture — indispensable quand une équipe grandit ou se renouvelle.

```
• • • DOCS/ADR/ADR-003-RESILIENCE.MD (TEMPLATE)  
  
# ADR-003 – Adoption de Resilience4j  
  
**Date : ** 2025-03-15  
**Statut : ** Accepté  
**Décideurs : ** Alice (tech lead), Bob (architecte), Carol (SRE)  
  
## Contexte  
loan-service appelle catalog-service de façon synchrone (HTTP).  
Des pannes intermittentes provoquent des timeouts en cascade.  
  
## Décision  
Adopter Resilience4j pour tous les appels inter-services synchrones.  
  
## Alternatives considérées  
| Alternative      | Raison d'exclusion |  
|-----|-----|  
| Netflix Hystrix  | Abandonné depuis 2018 |  
| Custom / manuel | Complexité, réinventer la roue |  
  
## Conséquences  
[+] Circuit breaker, retry, timeout disponibles avec annotations  
[+] Intégration native Spring Boot + Actuator + Micrometer
```

[...] CompletableFuture requis pour TimeLimiter (changement API)

[...] Courbe d'apprentissage sur l'ordre des décorateurs

4.2 Changements rétro-compatibles vs BREAKING

● Intermédiaire

✓ Rétro-compatibles

- Ajouter un champ optionnel dans la réponse
- Ajouter un nouveau endpoint
- Ajouter un paramètre optionnel
- Déprécier (mais garder) un champ

✗ BREAKING changes

- Supprimer ou renommer un champ
- Changer le type d'un champ
- Changer la sémantique d'un endpoint
- Rendre obligatoire un champ optionnel

```
LOANCONTROLLER.JAVA - GESTION VERSIONS API

@RestController
@RequestMapping("/api")
public class LoanController {

    // v1 maintenu pour compatibilité ascendante
    @GetMapping("/v1/loans/{id}")
    @Deprecated
    public LoanDtoV1 getLoanV1(@PathVariable UUID id) {
        return LoanMapper.toV1(loanService.findById(id));
    }

    // v2 - nouvelle structure enrichie
    @GetMapping("/v2/loans/{id}")
    public LoanDtoV2 getLoanV2(@PathVariable UUID id) {
        return LoanMapper.toV2(loanService.findById(id));
    }

    // V1 - structure originale
    public record LoanDtoV1(UUID id, String bookTitle, String studentName) {}

    // V2 - enrichi, sans casser les clients V1 (ils ignorent les nouveaux champs)
    public record LoanDtoV2(
        UUID id,
        String bookTitle,
        String studentName,
        LocalDate dueDate,      // nouveau
        LoanStatus status,      // nouveau
        List<Link> links         // HATEOAS
    ) {}
```

5. Urbanisation SI et choix d'architecture selon le contexte

Mode pro

Un bon architecte ne vend pas une architecture "préférée". Il choisit selon le contexte : taille de l'équipe, criticité, budget, maturité ops, charge, réglementation, dette. Il n'y a rarement qu'une seule bonne réponse.

Contexte	Choix souvent pertinent	Pourquoi
Startup / équipe < 5 dev	Monolithe modulaire	Vitesse, simplicité, pas d'overhead ops
Domaine stable, fort volume	Monolithe bien structuré	Cohérence transactionnelle, coût ops faible
Plusieurs équipes autonomes	Microservices ciblés	Découplage organisationnel (Loi de Conway)
Système très événementiel	Event-driven architecture	Découplage temporel, scalabilité async
Legacy à moderniser	Strangler Fig Pattern	Migration progressive, sans big bang risqué

5.1 Loi de Conway

● Intermédiaire

"Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure." — Melvin Conway, 1967

En pratique : si vous avez 3 équipes (catalog, loan, notification), vous aurez naturellement 3 services. Inversement, imposer des microservices à une de 3 personnes crée de la friction sans valeur.

5.2 Strangler Fig Pattern

● Expert

Moderniser un legacy en extrayant progressivement des features vers de nouveaux services, en plaçant un API Gateway devant les deux. À terme, le est entièrement remplacé sans jamais avoir fait de réécriture complète risquée.

Étape 1 : API Gateway devant le legacy
Gateway ---> [Legacy monolithe]

Étape 2 : Extraire une feature
Gateway ---> [Nouveau service A] (nouvelle feature)
 ---> [Legacy monolithe] (reste du système)

Étape N : Legacy entièrement remplacé
Gateway ---> [Service A] [Service B] [Service C]

5.3 Framework de décision

Dimension	Questions clés	Impact
Équipe	Taille ? Maturité DevOps ?	Microservices = overhead ops élevé
Domaine	Stable ou en rapide évolution ?	Stabilité → monolithe
Charge	Pics de trafic ? Scaling partiel ?	Scaling sélectif → microservices
Contraintes	RGPD, données médicales ?	Isolation et audit log
Budget	Coût infra ? TTM ?	Microservices = coût opérationnel élevé

6. Clôture — Synthèse et préparation à l'évaluation

📦 Livrable final attendu

1. Code TP3 (Spring Boot microservices) sur Git
2. 3 ADR minimum (choix d'architecture, résilience, sécurité)
3. Dossier d'architecture synthétique (2–3 pages) : contexte, décisions, compromis
4. Schéma des flux principaux (diagramme de séquence ou C4)
5. Justification des choix et limites connues du système

Ce que vous devez pouvoir expliquer à l'oral

- Jour 1 : Qu'est-ce qu'un bon découpage de domaine ? (SRP, DDD, bounded context)
- Jour 2 : Quand choisir un monolithe modulaire vs microservices ?
- Jour 3 : Différence entre REST, GraphQL, gRPC et quand choisir quoi ?
- Jour 4 : Communication sync vs async — avantages et inconvénients ?
- Jour 5 : Comment se défendre contre une cascade failure ?
- Jour 6 : Pourquoi "ça marche en dev mais pas en prod" ? (infra, secrets, sondes)

Q1 — "Pourquoi les microservices et pas un monolithe ?"

► [Ce qu'on attend](#)

Q2 — "Que se passe-t-il si catalog-service est indisponible 1h ?"

► [Ce qu'on attend](#)

Q3 — "Comment un nouveau dev déploie-t-il en production ?"

► [Ce qu'on attend](#)

Quiz de fin — Jour 6

1. Quelle est la différence entre Blue-Green et Canary ?

► [Voir la réponse](#)

2. Pourquoi utiliser un multi-stage build Docker ?

► [Voir la réponse](#)

3. Qu'est-ce qu'un ADR et pourquoi en écrire ?

► [Voir la réponse](#)

4. Qu'est-ce que la Loi de Conway ?

► [Voir la réponse](#)

5. (Expert) Qu'est-ce que le Strangler Fig Pattern ?[▶ Voir la réponse](#)[← Jour 5](#)